

Document de fin d'itération 3

Table des matières

1	Uniformité du code	1
1.1	Lint Checkstyle	1
1.2	Nommage	1
2	Listeners	1
2.1	View sans controller associé	2
3	Logique dans les Controllers	3
4	Basculement des écrans	4
5	Double dispatch et visitor	4
5.1	Application des effets	4
5.2	TowerController	5
6	Tâche pas terminée	5
7	Fonctionnalités	5
7.1	Génération Procédurale d'un étage	5
8	Estimation nombre d'heures	5

1 Uniformité du code

1.1 Lint Checkstyle

Nous avons ajouté celui-ci au début de cette itération. Nous avons rapidement vu les bénéfices: code beaucoup plus uniforme (même style).

1.2 Nommage

Nous avons été confrontés au problème de convention de nommage.

Le principe de 1 concept ↔ 1 nom n'était pas vraiment respectée.

Ce problème était déjà présent durant les itérations précédentes.

Au fur et à mesure de cette itération, nous avons ajouté des issues à chaque fois que nous voyions un problème de ce nommage.

Vers la fin de l'itération, nous avons :

1. écrit des conventions de nommage dans un document pour respecter le principe 1 concept ↔ 1 nom.
2. résolu toutes ces issues.

2 Listeners

- ajout des interfaces Listener dans toutes les Views.
- suppression de toutes les références au Controllers dans les Views.

Au départ les méthodes de nos Listeners ne prenaient jamais de paramètre. Lorsqu'une View signalait à son Listener (Controller) un événement, et que le Controller devait récupérer une information concernant l'événement, il utilisait un getter défini dans sa View.

Nous avons ensuite réalisé que ceci était mauvais. Notre solution consiste à passer les informations nécessaires directement en paramètre des méthodes de Listener.

Exemple :

```
// CreateBugemonView.java
public interface Listener {
    void onTypeSelected(BugemonType selectedType);

    void onAdd(String bugemonName, double healthValue, double attackValue,
                double defenseValue, double initiativeValue);
}
```

2.1 View sans controller associé

Les views qui sont des composants réutilisables n'ont pas de controller associé. Ainsi nous n'avons pas de controllers à définir comme étant les listeners de ces views.

Dans une configuration avec :

- Controller A
- View A
- ComponentView (un component de A)

Dans **View A**, nous instancions une classe anonyme pour jouer le rôle de Listener de **ComponentView**. Cette classe anonyme ne fait que déléguer la gestion d'évènements venant de **ComponentView** vers le listener de **View A** (i.e. **Controller A**).

Exemple :

Cette technique est utilisée à de nombreuses reprises dans ManualCombatView.

Cette classe contient plusieurs composants :

- ActionMenuView
- AttackMenuView
- SwitchMenuView
- ItemMenuView

Et celle-ci crée les listeners pour chacun de ces composants comme décrit ci-dessus. Voici un exemple pour la création du listener du component ActionMenuView dans CombatView:

```
// CombatView.java
private void initActionMenuViewListener() {
    this.actionMenu.setListener(new ActionMenuView.Listener() {

        @Override
        public void onAttack() {
            ManualCombatView.this.showAttackMenu();
        }

        @Override
        public void onSwitch() {
            ManualCombatView.this.showSwitchMenu(false);
        }

        @Override
        public void onInventory() {
            ManualCombatView.this.showInventory();
        }

        @Override
        public void onForfeit() {
            ManualCombatView.this.listener.onForfeit();
        }

    });
}
```

3 Logique dans les Controllers

Dans l'itération 2, nous avons de la logique dans les controllers, en particulier pour les fins de combats.

```
// CombatController
protected void onCombatEnded(Trainer winner) {
    this.playerService.restoreHpActiveTeam();

    List<LevelUp> levelUps = LevelUpService.distributeXpAndGetLevelUps(winner,
        this.combat.getOpponentTrainer());

    this.playerService.saveActiveTeamState();

    boolean won = winner == this.playerTrainer;
    this.metaController.onCombatFinished(levelUps, won);
}
```

Clairement, la distribution d'XP à la fin du combat, ainsi que la création des objets LevelUp (contenant les choix d'upgrade pour les montées de niveau) n'ont rien à faire dans un controller.

La distribution de l'XP (et la création des LevelUp) est une étape du déroulement d'un combat.

Notre solution :

L'objet Combat prend en paramètre un objet ICombatXpDistributor et demande à celui-ci de distribuer l'xp (délégation).

CombatXpDistributor calcule la quantité d'xp à distribuer aux bûgemoins et demande au BugemonService de distribuer l'xp.

BugemonService gère ensuite :

1. l'ajout d'xp sur le buggedon reçu en paramètre
2. la sauvegarde de l'état du buggedon (pour sauvegarder les nouveaux xp + niveau)
3. la création et le stockage interne des levelUps

La méthode Liste 3 devient alors :

```
// CombatController
protected void onCombatEnded(Trainer winner) {
    boolean won = winner == this.playerTrainer;
    this.metaController.onCombatFinished(won);
}
```

4 Basculement des écrans

Nous utilisons initialement une méthode *publique* `MetaController#switchTo(Window window)` pour gérer les transitions entre écrans.

Après avoir discuté Pr. Frédéric Pluquet, nous avons constaté que cette approche manquait de flexibilité. Nous avons adopté une solution plus adaptée qu'il nous a suggérée : ajouter au `MetaController` des méthodes lui permettant de réagir aux événements qui lui sont transmis par les autres contrôleurs.

Cette approche est notamment illustrée dans le `CombatController`, avec la méthode `onCombatEnded` (voir Liste 4).

Note : nous n'avons pas encore fini de remplacer tous les appels à `switchTo`. Ceci sera terminé au début de la prochaine itération.

5 Double dispatch et visitor

5.1 Application des effets

Utilisation du double dispatch pour éviter les `instanceof` dans l'application des effets sur les buggedons.

Buggedon expose une méthode :

```
// Buggedon.java
public void apply(Effet effect) {
    effect.applyTo(this); // dispatch vers le bon type effet
}
```

`Effet`, `EffectStatModifieur`, `EffectHeal`, `EffectResetMalus` ont tous la méthode suivante :

```
// this est un effet _concret_ ici
public void applyTo(Buggedon buggedon) {
    buggedon.apply(this); // appellera le bon overload
}
```

Ces méthodes `applyTo` permettent d'appeler le bon overload de la méthode `apply` dans `Buggedon` :

```
// Buggedon.java
public void apply(EffectStatModifieur e) { ... }
public void apply(EffectHeal e) { ... }
public void apply(EffectResetMalus e) { ... }
```

Note : maintenant que nous comprenons mieux le principe du double dispatch, nous avons remarqué plusieurs autres endroits où ce concept pourrait être utile dans la codebase. Notamment dans la logique du combat qui contient beaucoup de `instanceof`/switch sur le type.

5.2 TowerController

Nous utilisons également le *visitor pattern* pour la gestion des différents types concrets de Room.

TowerController implémente RoomVisitor.

6 Tâche pas terminée

Les items et l'inventaire ne sont pas encore stockés dans la base de données, bien qu'un membre du groupe ait ajouté cette fonctionnalité sur une branche du dépôt.

Note : cette fonctionnalité ne devait pas impérativement être implémentée dans cette itération, mais ayant fait le choix d'utiliser une base de données, et celle-ci gérant déjà la plupart des autres éléments du jeu (bugemons etc), il semblait cohérent d'ajouter la gestion des objets à celle-ci.

Malheureusement, dû à des problèmes de communication, cette fonctionnalité n'a pas été mergée. Celle-ci sera donc mergée dans l'itération 4.

7 Fonctionnalités

7.1 Génération Procédurale d'un étage

Un étage est structuré sous une forme d'arbre comprenant 3 à 4 sous-arbres générés aléatoirement. Chaque pièce de l'étage représentant un nœud FloorNode encapsulant les informations nécessaires (position, pièce, profondeur). Chaque étage a donc besoin de seulement un FloorNode représentant la racine/point de départ de l'étage.

Lorsqu'un controller fait appel à une pièce d'instance CombatRoom l'appel à `getCombat()` fait appel au CombatFactory ceci permettant une instanciation *lazy* d'un combat. De plus, le CombatFactory permet de prévoir de futures implémentation quant à la gestion des difficultés d'un étage ou d'un Boss.

8 Estimation nombre d'heures

Nous pensons avoir surestimé le nombre d'heures de l'histoire 16 (animation de déplacement).

Concrètement, nous avons terminé toutes les histoires de cette itération, mais un nombre d'heures non-négligeable a été consacré à du refactoring que nous n'avions pas mentionné au client.